CHAPITRE : STRUCTURES DE DONNÉES COMPLEXES

Introduction

Les structures de données complexes permettent d'organiser efficacement des données en mémoire afin de faciliter leur manipulation.

Dans ce chapitre, nous étudions principalement :

- Les **structures linéaires** :
    - listes contiguës
    - listes chaînées
    - piles
    - files
- Les **fichiers**

Une structure est dite **linéaire** lorsque chaque élément possède au plus un successeur.

1. Structures linéaires

1.1 Liste contiguë

1.1.1 Définition

Une **liste contiguë** est une structure de données où les éléments sont stockés **les uns à la suite des autres en mémoire**, dans un **tableau**. C'est donc une structure basée sur un tableau.

```
Type contig = Enregistrement
   Tab : tableau[0..s-1] de TypeElt
   lg  : entier
FinEnregistrement
```

- lg : nombre d'éléments
- s : taille maximale

Condition :

$0 \leq lg \leq s$

1.1.2 Insertion

```
Algorithme AJOUT(g, k, val)
Variables
  i, l : entier
Début
  l ← g.lg
  Si (l < s) ET (k ≤ l) Alors
    Pour i ← l-1 à k Pas -1 Faire
      g.Tab[i+1] ← g.Tab[i]
    FinPour
    g.Tab[k] ← val
    g.lg ← l + 1
  Sinon
    Ecrire("Insertion impossible")
  FinSi
Fin
```

1.1.3 Suppression

```
Algorithme SUPPRIME(g, k)
Variables
  i, l : entier
Début
  l ← g.lg
  Si k ≤ l - 1 Alors
    Pour i ← k à l - 2 Faire
      g.Tab[i] ← g.Tab[i+1]
    FinPour
    g.lg ← l - 1
  Sinon
    Ecrire("Suppression impossible")
  FinSi
Fin
```

1.2 Liste chaînée

1.2.1 Définition

Une liste chaînée représente un ensemble d'éléments organisés séquentiellement, tout comme un tableau. Sauf que dans un tableau, l'organisation séquentielle est donnée implicitement par la position dans le tableau. Dans une liste chaînée, on utilise un arrangement explicite dans lequel chaque élément ou maillon appartient à un nœud qui contient un lien au nœud suivant.
A la création, l'entête de la liste est toujours initialisé à NULL, puisque la liste est initialement vide.

```
Type Maillon = Enregistrement
  elt    : TypeElt
```

```
  suivant : ^Maillon
FinEnregistrement

Variable tete : ^Maillon
Début
  tete ← NIL
Fin
```

1.2.2 Création d'un maillon

```
Fonction CREER_MAILLON(val : TypeElt) : ^Maillon
Variable
  m : ^Maillon
Début
  Allouer(m)
  Si m ≠ NIL Alors
    m^.elt ← val
    m^.suivant ← NIL
  FinSi
  Retourner m
Fin
```

1.2.3 Insertion

a) En tête

```
Algorithme AJOUT_TETE(tete, val)
Variable
  m : ^Maillon
Début
  m ← CREER_MAILLON(val)
  Si m ≠ NIL Alors
    m^.suivant ← tete
    tete ← m
  FinSi
Fin
```

b) Au milieu

```
Algorithme AJOUT_MILIEU(pred, val)
Variable
  m : ^Maillon
Début
  m ← CREER_MAILLON(val)
  Si m ≠ NIL Alors
    m^.suivant ← pred^.suivant
    pred^.suivant ← m
  FinSi
```

```
Fin
```

c) En queue

```
Algorithme AJOUT_QUEUE(q, val)
Variable
   m : ^Maillon
Début
   m ← CREER_MAILLON(val)

   Si m ≠ NIL Alors
      m^.suivant ← NIL
      q^.suivant ← m
   FinSi
Fin
```

1.2.4 Suppression

a) En tête

```
Algorithme SUPPRIMER_TETE(tete)
Variable
   p : ^Maillon
Début
   Si tete ≠ NIL Alors
      p ← tete
      tete ← p^.suivant
      Libérer(p)
   FinSi
Fin
```

b) Au milieu

```
Algorithme SUPPRIMER_MILIEU(pred)
Variable
   p : ^Maillon
Début
   Si pred^.suivant ≠ NIL Alors
      p ← pred^.suivant
      pred^.suivant ← p^.suivant
      Libérer(p)
   FinSi
Fin
```

c) En queue

```
Algorithme SUPPRIMER_QUEUE(pred)
Variable
```

```
  p : ^Maillon
Début
  Si pred^.suivant ≠ NIL Alors
    p ← pred^.suivant
    pred^.suivant ← NIL
    Libérer(p)
  FinSi
Fin
```

1.2.5 Applications

Longueur de la liste

```
Fonction LONGUEUR(tete) : entier
Variables
  n : entier
  p : ^Maillon
Début
  n ← 0
  p ← tete
  TantQue p ≠ NIL Faire
    n ← n + 1
    p ← p^.suivant
  FinTantQue
  Retourner n
Fin
```

Saisie des éléments

```
Algorithme SAISIE(tete, n)
Variables
  i : entier
  val : TypeElt
Début
  Pour i ← 0 à n - 1 Faire
    Lire(val)
    AJOUT_TETE(tete, val)
  FinPour
Fin
```

1.3 Liste doublement chaînée

1.3.1 Définition

Une liste doublement chaînée est une liste qu'on peut parcourir dans les deux sens ; du premier élément au dernier et inversement. Ce qui impose l'ajout d'un autre champ à la structure du maillon qu'on appellera precedent et qui doit contenir l'adresse de son prédécesseur.

```
Type MaillonD = Enregistrement
   elt      : TypeElt
   suivant   : ^MaillonD
   precedent : ^MaillonD
FinEnregistrement
Variable tete : ^MaillonD ← NIL
```

1.3.2 Insertion en tête

```
Algorithme AJOUT_TETE_D(tete, val)
Variable
   m : ^MaillonD
Début
   Allouer(m)
   Si m ≠ NIL Alors
      m^.elt ← val
      m^.suivant ← tete
      m^.precedent ← NIL
      Si tete ≠ NIL Alors
         tete^.precedent ← m
      FinSi
      tete ← m
   FinSi
Fin
```

1.3.3 Suppression au milieu

```
Algorithme SUPPRIMER_MILIEU_D(p)
Début
   p^.precedent^.suivant ← p^.suivant
   p^.suivant^.precedent ← p^.precedent
   Libérer(p)
Fin
```

1.4 Piles

1.4.1 Définition (LIFO)

Une pile est une structure de données où les insertions et les suppressions se font toutes du même côté. Une telle structure est aussi appelée LIFO (Last In First Out).

```
Type ElementPile = Enregistrement
   elt  : TypeElt
   next : ^ElementPile
FinEnregistrement


Variable sommet : ^ElementPile ← NIL
```

1.4.2 Empiler

```
Algorithme EMPILER(sommet, val)
Variable
   e : ^ElementPile
Début
   Allouer(e)
   Si e ≠ NIL Alors
      e^.elt ← val
      e^.next ← sommet
      sommet ← e
   FinSi
Fin
```

1.4.3 Dépiler

```
Algorithme DEPILER(sommet)
Variable
   p : ^ElementPile
Début
   Si sommet ≠ NIL Alors
      p ← sommet
      sommet ← sommet^.next
      Libérer(p)
   FinSi
Fin
```

1.5 Files

1.5.1 Définition (FIFO)

Dans le cas d'une file, on fait les adjonctions (insertions) à une extrémité, les accès et les suppressions de l'autre extrémité. Les files sont aussi appelées FIFO (First In First Out). En d'autres termes, l'accès à un élément de la file suit le raisonnement d'une file d'attente « premier venu premier servi »

```
Type ElementFile = Enregistrement
   elt  : TypeElt
   lien : ^ElementFile
FinEnregistrement

Variables

   premier : ^ElementFile ← NIL

   dernier : ^ElementFile ← NIL
```

1.5.2 Enfiler

```
Algorithme ENFILER(premier, dernier, val)
Variable
  e : ^ElementFile
Début
  Allouer(e)
  Si e ≠ NIL Alors
    e^.elt ← val
    e^.lien ← NIL

    Si dernier = NIL Alors
      premier ← e
      dernier ← e
    Sinon
      dernier^.lien ← e
      dernier ← e
    FinSi
  FinSi
Fin
```

1.5.3 Défiler

```
Algorithme DEFILER(premier, dernier)
Variable
  p : ^ElementFile
Début
  Si premier ≠ NIL Alors
    p ← premier
    premier ← premier^.lien
    Libérer(p)
    Si premier = NIL Alors
      dernier ← NIL
    FinSi
  FinSi
Fin
```

2. Fichiers

2.1 Notion de fichier

Un **fichier** est une structure de données qui permet de **stocker des informations de manière permanente** sur un support (disque dur, SSD, clé USB...). Contrairement aux variables ou tableaux en mémoire, les données d'un fichier **persistent même après l'arrêt du programme.**

2.2 Ouverture

Ouvrir(fichier, "chemin", Lecture)

Ouvrir(fichier, "chemin", Ecriture)

2.3 Fermeture

Fermer(fichier)

2.4 Lecture

LireFichier(fichier, variables)

2.5 Écriture

EcrireFichier(fichier, données)

2.6 Exemple complet

Écriture

```
Algorithme ECRITURE_FICHIER
Variables
  nom, tel : chaîne
  i : entier
Début
  Ouvrir(f, "personne.txt", Ecriture)
  Pour i ← 1 à 10 Faire
    Lire(nom)
    Lire(tel)
    EcrireFichier(f, nom, " ", tel)
  FinPour
  Fermer(f)
Fin
```

Lecture

```
Algorithme LECTURE_FICHIER
Variables
  nom, tel : chaîne
Début
  Ouvrir(f, "personne.txt", Lecture)
  TantQue NON FinDeFichier(f) Faire
    LireFichier(f, nom, tel)
    Ecrire(nom, tel)
  FinTantQue
```

```
    Fermer(f)
Fin
```

Exercices Listes chainées

Exercice 1 : Compréhension

On a la liste :

10 → 20 → 30 → 40 → NIL

1. Que devient la liste après insertion de 5 en tête ?
2. Que devient la liste après suppression du premier élément ?

Corrigé Exercice 1

1. Après insertion en tête : 5 → 10 → 20 → 30 → 40 → NIL
2. Après suppression : 20 → 30 → 40 → NIL

Exercice 2 : Insertion au milieu

On a :

10 → 20 → 40 → NIL

Insérer 30 après 20

Corrigé Exercice 2

10 → 20 → 30 → 40 → NIL

Exercice 3 : Parcours

Écrire un algorithme qui affiche tous les éléments d'une liste chaînée.

Corrigé Exercice 3

```
Algorithme AFFICHER(tete)
Variable
  p : ^Maillon
Début
  p ← tete
  TantQue p ≠ NIL Faire
    Ecrire(p^.elt)
    p ← p^.suivant
  FinTantQue
Fin
```

Exercice 4 : Recherche

Écrire un algorithme qui vérifie si un élément existe dans une liste chaînée.

Corrigé Exercice 4

```
Fonction RECHERCHE_LC(tete, val) : booléen
Variable
  p : ^Maillon
Début
  p ← tete
  TantQue p ≠ NIL Faire
    Si p^.elt = val Alors
      Retourner VRAI
    FinSi
    p ← p^.suivant
  FinTantQue
  Retourner FAUX
Fin
```

Exercices Liste doublement chaînée

Exercice 5 : Compréhension

On a :

10 ⇄ 20 ⇄ 30 ⇄ NIL

1. Ajouter 5 en tête
2. Supprimer 20

Corrigé Exercice 5

1. Ajout en tête :

5 ⇄ 10 ⇄ 20 ⇄ 30 ⇄ NIL

2. Suppression de 20 :

5 ⇄ 10 ⇄ 30 ⇄ NIL

Exercice 6 : Algorithme

Écrire un algorithme qui parcourt la liste dans les deux sens.

Corrigé

```
Algorithme PARCOURS_DOUBLE(tete)
Variables
  p : ^MaillonD
  fin : ^MaillonD
Début
  p ← tete
  // Aller vers la fin
  TantQue p^.suivant ≠ NIL Faire
    Ecrire(p^.elt)
    p ← p^.suivant
  FinTantQue
  fin ← p
  Ecrire(fin^.elt)

  // Retour arrière
  TantQue fin ≠ NIL Faire
    Ecrire(fin^.elt)
    fin ← fin^.precedent
  FinTantQue
Fin
```

Exercices Piles

Exercice 7 : Simulation

Effectuer les opérations suivantes :

Empiler(10)

Empiler(20)

Empiler(30)

Dépiler()

Empiler(40)

Corrigé

Étapes :

Pile :

10

20

30

Après dépiler :

10

20

Après empiler 40 :

10

20

40

Sommet = 40

Exercice 8 : Vérification pile vide

Écrire un algorithme qui vérifie si une pile est vide.

Corrigé

```
Fonction EST_VIDE(sommet) : booléen
Début
  Si sommet = NIL Alors
    Retourner VRAI
  Sinon
    Retourner FAUX
  FinSi
Fin
```

Exercice 9 : Compter les éléments

Écrire un algorithme qui compte le nombre d'éléments d'une pile.

Corrigé

```
Fonction TAILLE_PILE(sommet) : entier
Variables
  p : ^ElementPile
  n : entier
Début
  n ← 0
  p ← sommet

  TantQue p ≠ NIL Faire
    n ← n + 1
```

```
    p ← p^.next
  FinTantQue

  Retourner n
Fin
```

Exercices Files

Exercice 10 : Simulation

Effectuer :

Enfiler(10)

Enfiler(20)

Enfiler(30)

Défiler()

Enfiler(40)

Corrigé

File :

10 20 30

Après défiler :

20 30

Après enfiler 40 :

20 30 40

Exercice 11 : Test file vide

Corrigé

```
Fonction FILE_VIDE(premier) : booléen
Début
  Retourner (premier = NIL)
Fin
```

Exercices Fichiers

Exercice 12 : Écriture

Écrire un algorithme qui enregistre 5 noms dans un fichier.

Corrigé

```
Algorithme ECRITURE_NOMS
Variables
  nom : chaîne
  i : entier
Début
  Ouvrir(f, "noms.txt", Ecriture)

  Pour i ← 1 à 5 Faire
    Lire(nom)
    EcrireFichier(f, nom)
  FinPour

  Fermer(f)
Fin
```

Exercice 13 : Lecture

Afficher le contenu du fichier.

Corrigé

```
Algorithme LECTURE_NOMS
Variables
  nom : chaîne
Début
  Ouvrir(f, "noms.txt", Lecture)

  TantQue NON FinDeFichier(f) Faire
    LireFichier(f, nom)
    Ecrire(nom)
  FinTantQue

  Fermer(f)
Fin
```

Exercice 14 (classique examen)

Inverser une liste chaînée

Corrigé

```
Algorithme INVERSER_LISTE(tete)
Variables
   prev, curr, next : ^Maillon
Début
   prev ← NIL
   curr ← tete

   TantQue curr ≠ NIL Faire
      next ← curr^.suivant
      curr^.suivant ← prev
      prev ← curr
      curr ← next
   FinTantQue

   tete ← prev
Fin
```

Sources : *https://elearning.esgee-oran.dz/pluginfile.php/16193/mod_page/content/68/fzlebbah-support-cours.pdf*